

PRODUCT & ARCHITECTURE HANDBOOK

Enterprise AI-Driven HRMS Platform

A configurable, multi-tenant, AI-native
People Operations Platform for the India-
first mid-market

Multi-Tenant

No-Code Configurable

AI / RAG-Native

Payroll & Compliance

Event-Driven

White-Label

Working title: Enterprise AI-Driven HRMS Platform

Prepared from: 41 Approved project documents

Owner: Bhajan Lal | **Status:** Approved material as of 2026-06-27

Acronym: HRMS = Human Resource Management System

About This Handbook

This document explains **what we are building and why**, in plain language, for anyone new to the project — leadership, new team members, partners, or reviewers.

HOW TO READ THIS

It is ordered the way the project itself was decided: **Vision** → **Market** → **Product** → **Roadmap** → **Architecture** → **Decisions** → **Database** → **Security** → **UI** → **API** → **AI**. You can read it top to bottom without prior knowledge. The first time a short form (abbreviation) appears, its full form is given in brackets — for example, *API (Application Programming Interface)*. A full glossary appears at the end.

What "Approved" means here

This project follows a strict rule: **no code is written before documentation is approved**. A document is "Approved" only when its owner (Bhajan Lal) signs it off. This handbook is built *exclusively* from the 41 documents that currently carry that approved status, so everything here is settled direction, not speculation.

Document control

Field	Value
Title	Enterprise AI-Driven HRMS Platform — Product & Architecture Handbook
Source	41 Approved documents in the project docs/ tree + project constitution
Prepared	2026-06-27
Audience	Executives, new joiners, partners, reviewers (non-technical friendly)
Confidentiality	Internal — contains product strategy
Nature	Explanatory summary. The approved source documents remain the authoritative record.

Contents

1. Executive Summary	p.3
2. Vision & Mission	—
3. The Market & Why We Will Win	—
4. What the Product Does (Requirements)	—
5. Delivery Roadmap	—
6. System Architecture	—
7. Key Decisions (ADRs)	—
8. Database Foundations	—

9. Security & Tenant Isolation	—
10. User Interface & Experience	—
11. API Design	—
12. AI Strategy	—
13. Glossary of Terms	—
14. Index of All 41 Approved Documents	—

1. Executive Summary

We are building an enterprise-grade **HRMS (Human Resource Management System)** — software that runs a company's entire employee lifecycle: hiring, attendance, leave, payroll, performance, and exit.

Most HRMS products force companies to **raise a support ticket and wait for the vendor** every time they want to change a policy, an approval flow, or a form. Our core bet is to remove that friction: HR teams configure the platform themselves, safely, with no code and no waiting.

THE ONE-SENTENCE POSITIONING

A configurable **People Operations Platform** that lets growing companies change HR policies, workflows, payroll rules, reports, and employee journeys **without vendor tickets** — with payroll-grade compliance, audit, rollback, and AI assistance.

The five things that make this different

Pillar	What it means in plain words
Compliance trust	Indian statutory payroll done correctly — PF, ESI, PT, LWF, TDS, Form 16, Gratuity, Full & Final Settlement (all expanded in §3). This is the price of entry in India.
No-code configurability	HR changes approval flows, forms, rules and reports themselves — our "Workflow Studio" is the headline feature.
Mid-market usability	Clean, mobile-first, low training cost — usable by non-technical HR and busy managers.
Full-suite breadth, delivered in phases	Recruitment, performance, learning and more are on the roadmap, added as plug-in modules without rewriting the core.
AI-native but grounded	An AI assistant that answers from <i>your</i> approved policies, cites its sources, and never bypasses security.

Who it is for

The **underserved 50–2,000 employee Indian mid-market** — too big for cheap startup tools, too cost-sensitive and fast-moving for heavyweight global enterprise suites. The architecture also supports multi-country expansion (UAE, USA, UK) through plug-in compliance packs.

Where the project is today

The project is in its **documentation-first phase**. Research, product requirements, roadmap, architecture, database foundations, security design, UI design, API specifications, and AI strategy are **Approved**. Detailed AI operations, per-feature specifications, and the first build sprint are the next steps. **41 documents are approved** to date.

2. Vision & Mission

Mission

Build a configurable, multi-tenant, enterprise HRMS platform that competes with greythHR, Keka, HROne, Zoho People, Darwinbox, BambooHR, Workday, SAP SuccessFactors and UKG.

The word **multi-tenant** means one single software installation safely serves many separate customer companies at once — each customer is a "tenant", and no tenant can ever see another tenant's data. (This is explained fully in §6 and §9.)

The platform must support

- **Multi-Tenant** — many customers, one platform
- **Multi-Country** — India first, then UAE/USA/UK
- **Multi-Language** — English + Hindi at launch
- **White Labeling** — each customer's own logo/colours/domain
- **Dynamic Workflows** — approval flows built without code
- **Dynamic Forms** — custom fields without code
- **Dynamic Rules** — business rules stored as data
- **Dynamic Reports** — customer-defined reports
- **Extension-Based Customization** — add-ons, not core edits
- **AI Features** — grounded, permission-aware assistance

The Golden Rules (the project's constitution)

Every contributor — human or AI agent — must follow ten non-negotiable rules. These are the project's DNA:

Rule	In plain words
1. Docs first	No code before approved documentation (five docs per feature: Business, Technical, Database, UI, Test).
2. No hardcoded rules	Never write <code>if (leaveDays > 5)</code> in code. Rules live in a database / rules engine, so they are configurable.
3. Customisation never touches core	Customer-specific needs are met by configuration, feature flags, extensions, and plugins — never by editing core logic.
4. RBAC + ABAC + Audit everywhere	Every module enforces role-based and attribute-based access control, and logs everything (defined in §6/§9).
5. Every API versioned, documented, tested	OpenAPI documentation is mandatory (§11).

6. Event-driven where possible	Modules talk through events, e.g. <i>Attendance Approved</i> → <i>Payroll</i> → <i>Notifications</i> → <i>Reports</i> .
7. Tenant isolation	Every table has a <code>TenantId</code> ; every query is tenant-filtered. No cross-tenant access, ever.
8. Decisions documented	Every architectural fork becomes an ADR (Architecture Decision Record) — see §7.
9. SOLID	Code follows SOLID, DRY, KISS, YAGNI, Clean Architecture and DDD (Domain-Driven Design) principles.
10. Test coverage ≥ 85%	Plus integration and end-to-end tests.

WHY RULE 2 MATTERS — A CONCRETE EXAMPLE

Suppose a customer's policy is "sick leave over 5 days needs HR approval". A normal product hardcodes the number `5`. When the government or the company changes the threshold, an engineer must edit and redeploy code. In our platform the `5` is a **rule stored as data**. An HR admin edits it in the screen, it takes effect immediately, the old version is preserved with history — and no engineer is involved.

3. The Market & Why We Will Win

Source: Approved competitor analysis (India-first) and gap analysis. India is the launch market.

The market is split into four bands

Segment	Employees	What they buy on	Who serves them today
Micro / Startup	1–50	Cheap, fast payroll	Kredily, RazorpayX, Zoho
SMB (<i>Small & Medium Business</i>)	50–500	Payroll + compliance + ease	greytHR, Keka, HROne
Mid-Market	500–2,000	Configurable workflows, integrations	Keka, Darwinbox, HROne
Enterprise	2,000+	Multi-entity, global, deep config	Darwinbox, SAP, Workday, UKG

THE OPENING WE ARE TARGETING

The **50–2,000 employee band** is the most contested and the most underserved on one specific thing: **configurability without code**. Cheap tools hit a "configuration ceiling"; enterprise tools are powerful but expensive and slow to implement. We sit in the middle: *enterprise-grade architecture, mid-market speed and price*.

What customers actually complain about

Crucially, research showed customers do **not** churn because a module is missing. They churn because of **friction**:

- "No-code" tools that still need vendor tickets for real changes.
- Payroll edge cases (FBP, mid-cycle CTC changes, arrears) that go manual every month.
- Slow support, especially during payroll week.
- Updates that cause unexpected glitches ("regressions").
- Mobile/attendance reliability problems and punch-sync failures.
- Reporting that always falls back to Excel.

FBP = Flexible Benefits Plan · CTC = Cost To Company (total salary package).

Our differentiators (the moat)

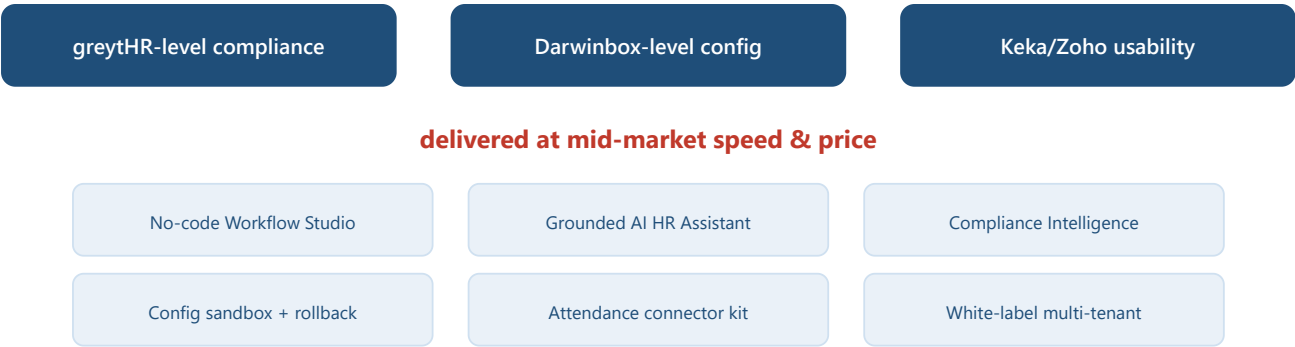


Figure 3.1 — Our position combines three competitor strengths at a price/speed they cannot match, on top of platform capabilities (lower row).

The wedge in one line: a **no-code HR operating system with compliance-safe change management** — HR can change leave policies, payroll rules, approvals, forms, roles, reports and branding without vendor tickets, code deployments, lost audit history, or cross-tenant risk.

4. What the Product Does (Requirements)

Source: Approved PRD (Product Requirements Document). It defines **15 functional requirements**, labelled FR-001 to FR-015.

READING TIP

"FR" = Functional Requirement. The **numbers are just stable IDs, not the build order**. For example FR-015 has the highest number but is built *first*, because nothing else is safe to build until tenant isolation exists. **"Must"** = required for the first delivery phase; **"Should"** = important but can follow.

ID	Requirement	What it delivers, in plain words	Priority
FR-001	Multi-tenant foundation	Every record belongs to one tenant; modules can be switched on/off per customer without redeploying.	Must
FR-002	Identity, sign-in & access control	Secure login (JWT), plus RBAC + ABAC so people see only what their role and attributes allow.	Must
FR-003	Core HR & Employee Master + ESS	Employee records, org structure, documents, and Employee Self-Service (ESS) profile editing.	Must
FR-004	Leave Management	Configurable leave types, balances, accrual and holidays, with multi-level approval — no hardcoded rules.	Must
FR-005	Attendance & device connectors	Attendance via web/mobile/manual/CSV plus plug-and-play biometric devices (e.g. ZKTeco, eSSL).	Must
FR-006	Payroll + India compliance	Salary structures, formula engine, payslips, and all Indian statutory outputs (PF, ESI, PT, LWF, TDS, Form 16).	Must
FR-007 ★	Workflow Studio (no-code engine)	The headline differentiator. Visual builder for approval chains, conditions, forms and notifications, reused across every module.	Must
FR-008	Audit & "time machine"	Every change (create/update/delete/login/approval) recorded with before/after values, who, when, why.	Must
FR-009	Notifications & event bus	Events such as <i>LeaveApproved</i> trigger notifications and downstream actions without tight coupling.	Must
FR-010	White-label & branding	Per-tenant logo, colours, custom domain and email templates — from configuration only.	Should
FR-011	RAG HR Assistant	Employees ask plain-language questions, answered from the company's own policies with citations.	Should
FR-012	Reporting & exports	Standard payroll/leave/attendance reports with filters and CSV/Excel export.	Should
FR-013 ★	Business Rule Engine	A generic "rules-as-data" engine used by Leave, Payroll, routing and validation — versioned and effective-dated.	Must
FR-014 ★	Effective-dated / bitemporal core	Supports past-dated corrections and future-dated changes with full history and "as-of" queries.	Must

FR-015 ★	Tenant Catalog + Row-Level Security	The foundation that makes tenant isolation bullet-proof — built first.	Must
-------------	-------------------------------------	--	------

WORKED EXAMPLE — FR-014 "EFFECTIVE-DATED" IN REAL LIFE

On 1 April an employee gets a raise effective **1 July**. The system stores it now but it only takes effect in July automatically. Separately, HR discovers a March salary was wrongly entered — they make a **back-dated correction** without erasing what was there before. Months later finance asks "what was this person's salary **as of 15 March?**" — the system answers exactly, because it never overwrites history. Most cheaper tools cannot do this; it is a Workday-grade strength built into our core.

Beyond the first phase

The product is a full suite delivered over phases. Later modules — Recruitment/ATS (Applicant Tracking System), Onboarding, Performance/OKRs (Objectives & Key Results), LMS (Learning Management System), Engagement, HR Service Desk, Documents/e-sign, Shifts, Expenses, Assets, Compensation review and more — all plug in through the same engines. **The rule:** new modules are added by configuration and extension, never by changing existing core code.

5. Delivery Roadmap

Source: Approved phased-delivery roadmap. The guiding principle is **foundations before features**.

The build sequence is driven by dependencies, not by feature popularity. You cannot safely build Leave or Payroll until the platform foundations (tenant isolation, identity, rules, workflow, audit, events) exist. So the foundations come first, even though they are invisible to the end user.

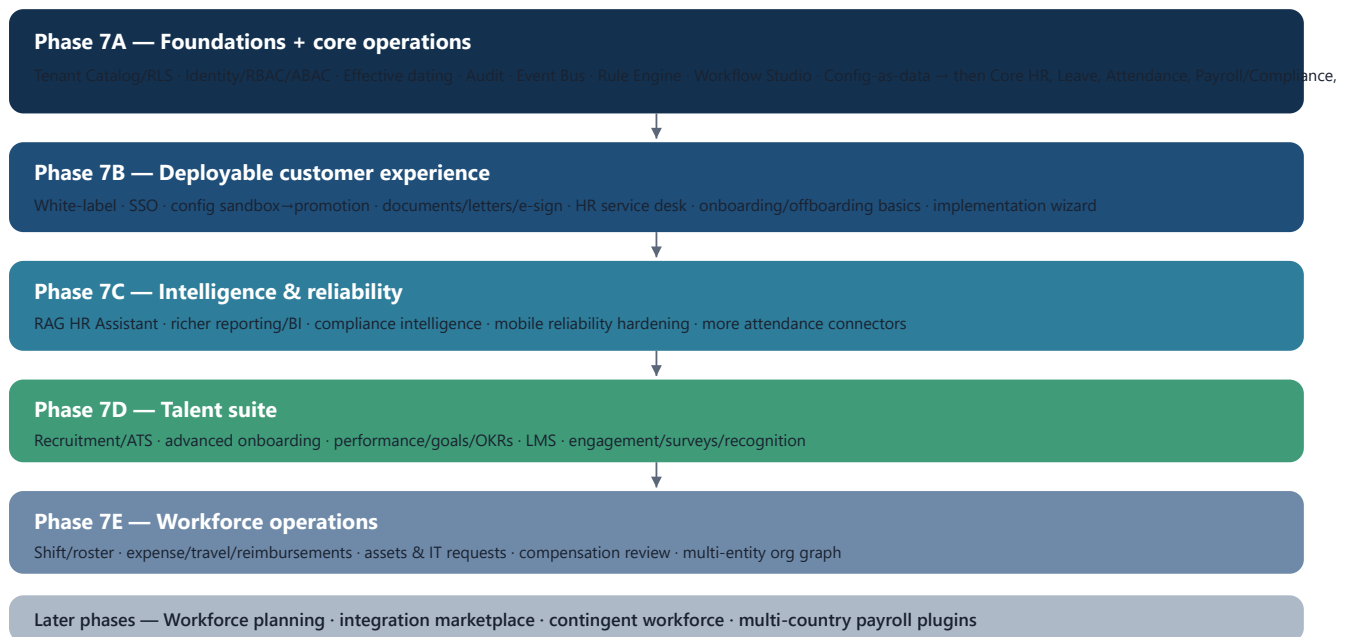


Figure 5.1 — Phased delivery. Each phase ships end-to-end vertical slices behind feature flags; later phases plug in without core rewrites.

The critical path (what blocks what)

The single most important build order is: **FR-015 Tenant Catalog/RLS** → **FR-002 Identity** → **FR-013 Rules & FR-007 Effective Dating** → **FR-006/FR-004 modules**. Workflow Studio (FR-007) is both the biggest differentiator *and* the biggest technical risk, so it is prototyped early and hardened throughout Phase 7A.

THE HARD GATE

No build starts on any feature until its **five documents** (Business, Technical, Database, UI, Test) are all Approved. And no phase begins until the previous phase's required documents are Approved. This discipline is what protects payroll customers from breaking changes.

6. System Architecture

Source: Approved architecture review, provider-agnostic review, and foundational ADRs.

The shape: a Modular Monolith

The system is built as a **Modular Monolith** — one deployable application, internally divided into clean, independent modules (Employee, Leave, Payroll, etc.). This keeps early development simple and fast, while the clean module boundaries mean individual modules can later be split into separate **microservices** if scale demands it. It is a deliberate "start simple, stay flexible" choice (recorded in ADR-004).

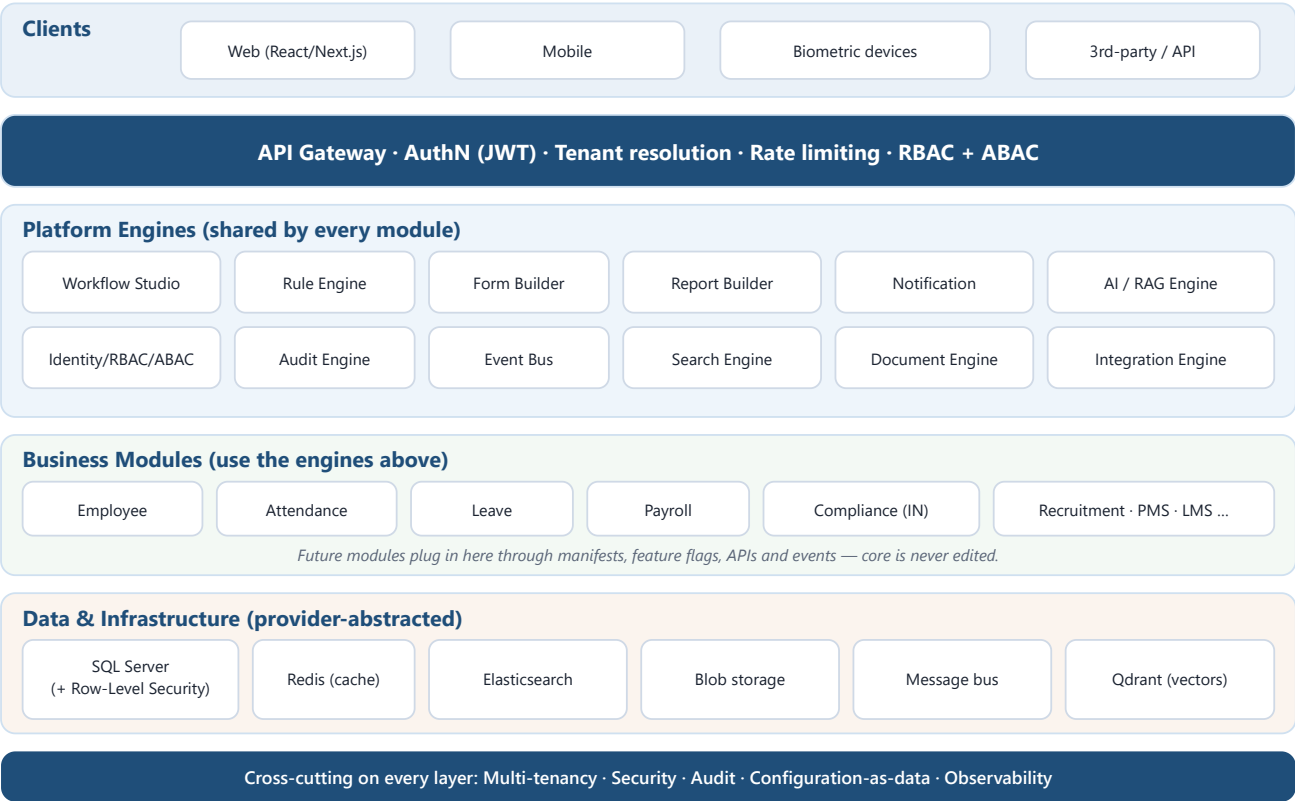


Figure 6.1 — Layered architecture. Business modules never re-implement security, workflow, or rules; they consume shared platform engines.

Two access-control concepts you will see everywhere

RBAC — ROLE-BASED ACCESS CONTROL

Permissions follow your **role**. An "HR Admin" can run payroll; an "Employee" cannot. Simple and familiar.

ABAC — ATTRIBUTE-BASED ACCESS CONTROL

Permissions also follow your **attributes**. A manager sees only employees *in their own department/location* — not the whole company. RBAC says "what you can do"; ABAC says "on whose data".

WORKED EXAMPLE — EVENT-DRIVEN DESIGN (RULE 6) IN ACTION

A manager approves a leave request. Instead of one giant function doing everything, the Leave module simply **publishes an event**: *LeaveApproved*. Independently, the Notification engine emails the employee, the Payroll module adjusts the records, the Audit engine logs it, and Reports update — all **without the Leave module knowing they exist**. New consumers can be added later without touching Leave. This is what "event-driven" and "loose coupling" mean.

7. Key Decisions (ADRs)

An **ADR (Architecture Decision Record)** is a short document capturing one important decision, the options considered, and why one was chosen — so future teams understand the "why", not just the "what". **19 ADRs are approved.**

ADR	Decision	Why it matters (plain words)
ADR-001	React / Next.js frontend	Modern, fast, widely-supported web UI technology.
ADR-002	.NET as primary backend	Enterprise-grade, high-performance backend platform.
ADR-003	SQL Server database	Reliable, well-understood relational database with strong security features (incl. RLS).
ADR-004	Modular Monolith	Start as one app with clean modules; split to microservices later if needed.
ADR-005	Multi-tenancy model	How tenants are kept separate, including Row-Level Security as a safety net.
ADR-006	Tenant context & data access	Tenant identity is resolved server-side and never trusted from the client.
ADR-007	Effective-dated / bitemporal data	How past-dated corrections and future-dated changes are stored with full history.
ADR-008	Identity & access	Login, single sign-on path, and how RBAC + ABAC are enforced.
ADR-009	Event-driven backbone	The event bus + "outbox" pattern that makes modules talk reliably and asynchronously.
ADR-010	Workflow engine	Build a lean interpreter that reads workflow definitions as data (configuration, not code).
ADR-011	Rule engine / expression language	Safe, sandboxed rules-as-data (no arbitrary code execution).
ADR-019	Enterprise AI / RAG architecture	The umbrella decision for the whole AI layer (see §12).
ADR-027	Provider-abstraction framework	Storage, cache, messaging, AI providers etc. sit behind interfaces, so vendors can be swapped without core changes.
ADR-030	Vector store strategy	Self-hosted Qdrant first (avoids paid search subscriptions); Azure AI Search optional later.
ADR-031	AI observability & telemetry	How AI behaviour is monitored, with privacy-safe metrics.
ADR-032	Conversation memory strategy	AI chat memory defaults to session-only; long-term memory is opt-in and encrypted.
ADR-033	AI cost governance	Budgets, quotas and limits so AI spend can never run away.
ADR-034	AI / RAG evaluation framework	How AI quality, fairness and safety are tested before release.

ADR-035	Semantic / retrieval cache	Self-hosted Redis cache for AI, with strict tenant/permission partitioning.
---------	----------------------------	---

THE THREAD CONNECTING THEM

Notice the pattern: nearly every decision protects two things — **tenant isolation** and **configurability**. Technology choices (ADR-001/002/003) are deliberately conventional; the *interesting* decisions are about keeping customers safely separated and letting them change behaviour without code.

8. Database Foundations

Source: Approved foundational, tenant-catalog, and identity database designs.

The database is split into clean **schemas** (named groups of tables) — Core, Employee, Attendance, Leave, Payroll, Workflow, Audit, Security — so it never becomes one giant tangle. Two foundations carry the most weight: the **tenant catalog** and the **identity** design.

How tenant separation works in the data

Every business table carries a `TenantId` column. But a column alone is not enough — a developer could forget to filter by it. So the design uses **three layers of defence**:

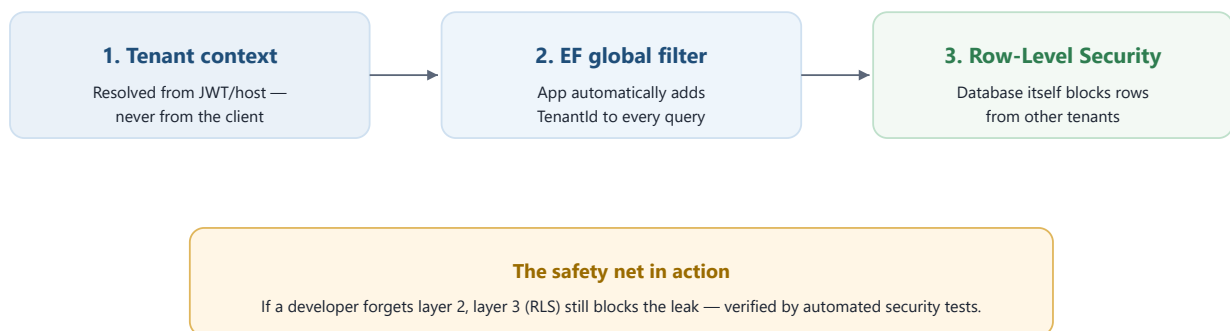


Figure 8.1 — Defence-in-depth for tenant isolation. RLS = Row-Level Security, enforced by the database engine itself.

The tenant catalog

A small separate "catalog" database maps each tenant to its **placement** (which database/shard and region it lives in), its **plan and entitlements** (which modules/features it has paid for), its **feature flags**, and its **branding**. This is what lets a tenant be moved to a dedicated database for data-residency reasons *by configuration*, with no code change.

The identity design

Stores users, roles, permissions and the attributes (department, location, business unit) that power ABAC. Every access decision — and every *denied* access decision — is auditable.

KEY PRINCIPLE — DERIVED VS AUTHORITATIVE DATA

Source records in SQL Server are the single source of truth. Search indexes, caches, and AI vectors are **derived** copies that can always be rebuilt. This rule keeps the system safe: if a cache is ever wrong, it is thrown away and regenerated from the authoritative database.

9. Security & Tenant Isolation

Source: Approved security design & threat model. The work uses **STRIDE** (a standard method for finding threats) and defends the **OWASP Top 10** (the industry list of the most common web vulnerabilities).

Five security objectives

- **Tenant isolation is absolute** — no cross-tenant read or write under any path.
- **Deny by default** — access is refused unless explicitly granted (RBAC + ABAC on every request).
- **Confidentiality of PII & payroll** — encryption and masking. PII = Personally Identifiable Information.
- **Tamper-evident audit** — who did what, when, on whose behalf — and it cannot be quietly altered.
- **OWASP Top 10 defended** by design and verified by tests.

Trust boundaries — nothing trusts the layer above blindly

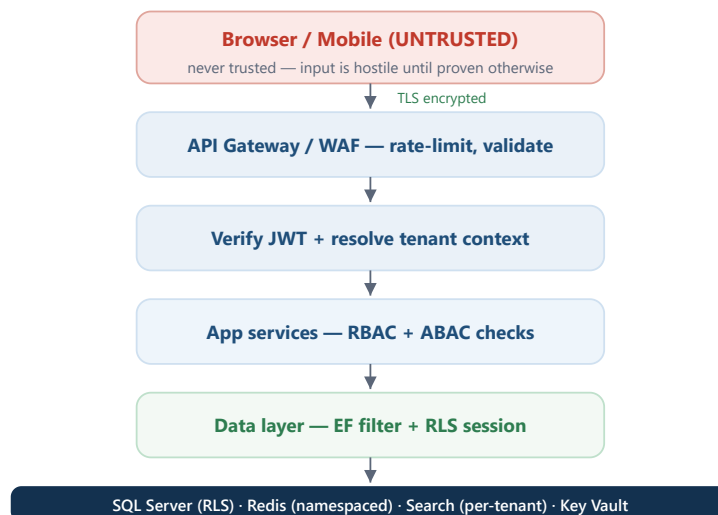


Figure 9.1 — Every inward hop re-establishes identity and tenant. JWT = JSON Web Token; WAF = Web Application Firewall; EF = Entity Framework; RLS = Row-Level Security.

STRIDE in plain words

Threat (STRIDE)	Example	Our control
Spoofing (faking identity)	Stolen or forged login token	Short-lived signed JWT, token rotation, MFA
Tampering (altering data)	Editing another tenant's record	TenantId + RLS, append-only audit with hash-chain
Repudiation ("I didn't do it")	Denying an approval	Immutable audit incl. "on behalf of", correlation IDs
Information disclosure	Cross-tenant data leak, PII in logs	RLS + global filter, field masking, log redaction
Denial of service	One tenant exhausts resources	Per-tenant rate limits/quotas, isolated workers
Elevation of privilege	Tenant admin tries to become platform admin	Role separation, least privilege, sandboxed rules (no code execution)

MFA = Multi-Factor Authentication. The threat model also adds a seventh boundary — provider abuse — controlled by allowlists and egress controls.

10. User Interface & Experience

Source: Approved design system, screen specifications, and the People-Ops design spec (with an interactive prototype). UI/UX = User Interface / User Experience.

Design principles

- **Mobile-first & responsive** — works on a phone first, scales up to desktop.
- **Accessibility compliant** — keyboard and screen-reader friendly (WCAG standards).
- **Low learning curve** — usable by non-technical HR and busy managers.
- **Role-based dashboards** — each person sees a screen built for their role, driven by metadata (configuration), not hardcoded pages.
- **AI woven in, not bolted on** — a persistent Copilot and a ⌘K command centre.

Signature configurability screens

The prototype proves the "no-code" promise with builder screens — not just data screens:

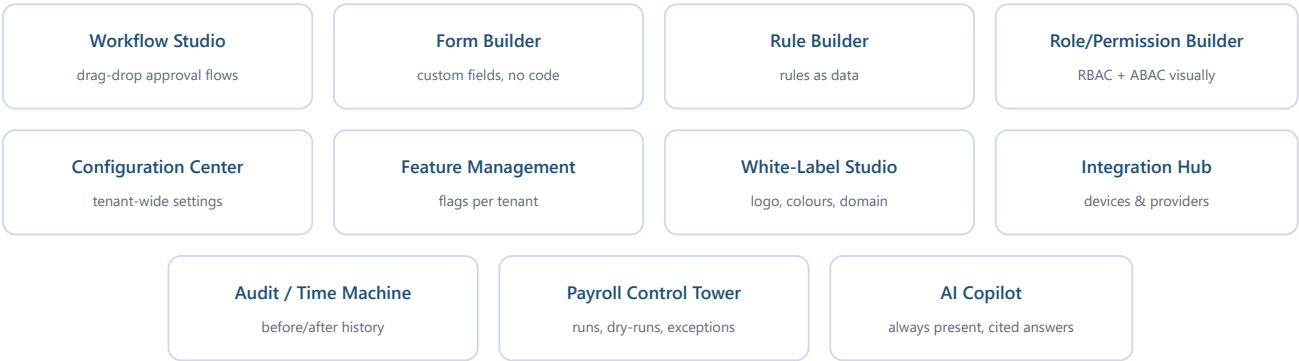


Figure 10.1 — The configurability screens. These "builder" surfaces are what let HR change the system without engineers.

WHY THIS MATTERS COMMERCIALY

Competitors often demo lovely *data* screens, then reveal that changing a workflow needs a support ticket. Our prototype deliberately leads with *builder* screens — the visible proof of the "no vendor tickets" promise that the whole product strategy rests on.

The owner reviewed and approved the design system, the five foundational screens (login/tenant, directory, apply-leave, approvals inbox, Workflow Studio), and the full design spec with prototype.

11. API Design

Source: Approved foundational API specification and machine-readable OpenAPI file (44 paths, 53 operations).

An **API (Application Programming Interface)** is the contract that lets the web app, mobile app, and outside systems talk to the backend in a defined, predictable way. **OpenAPI** is the industry-standard format for writing that contract down so it is documented, testable, and tool-friendly. Project rule: *every API is versioned, documented in OpenAPI, and tested.*

Conventions every endpoint follows

- **Versioned** — all paths begin `/api/v1/...` so future changes don't break existing callers.
- **Tenant + security enforced** — tenant is resolved server-side; RBAC + ABAC apply on every call.
- **Consistent envelope** — every response has the same shape (data + metadata + errors).
- **Pagination & idempotency** — large lists are paged; repeat-safe operations use idempotency keys so a retried request never double-acts.

WORKED EXAMPLE — APPLYING FOR LEAVE THROUGH THE API

Request

```
POST /api/v1/leave/requests
Authorization: Bearer <JWT> // tenant + user identity inside the token
Idempotency-Key: 7f3c-... // retry-safe
{
  "leaveType": "SICK",
  "from": "2026-07-10", "to": "2026-07-16",
  "reason": "Fever"
}
```

What happens server-side

1. Tenant + identity resolved from the JWT (never trusted from the body)
2. RBAC/ABAC: is this user allowed to apply leave for themselves?
3. Rule engine: "sick leave > 5 days → needs HR approval" (data, not code)
4. Workflow engine starts the configured approval chain
5. Event *LeaveRequested* published; audit recorded

Response 201 Created

```
{ "id": "...", "status": "PENDING_HR_APPROVAL", "approver": "..." }
```

The approved API set covers authentication, employees (effective-dated), leave (rules + workflow), workflow/approvals (with delegation and SLA), search, notifications, AI, and provider management. The provider-management endpoints align with the provider-abstraction decision (ADR-027), so customers can configure their own storage/search/notification providers.

12. AI Strategy

Source: Approved AI Strategy v2.1 and the AI/RAG ADR family (ADR-019, 030–035). AI = Artificial Intelligence.

The AI assistant answers HR, policy and payroll questions in plain language. The method is **RAG (Retrieval-Augmented Generation)**: instead of the AI making things up from general knowledge, it first *retrieves* the relevant passages from *your company's own approved documents*, then *generates* an answer grounded in them — **with citations**.

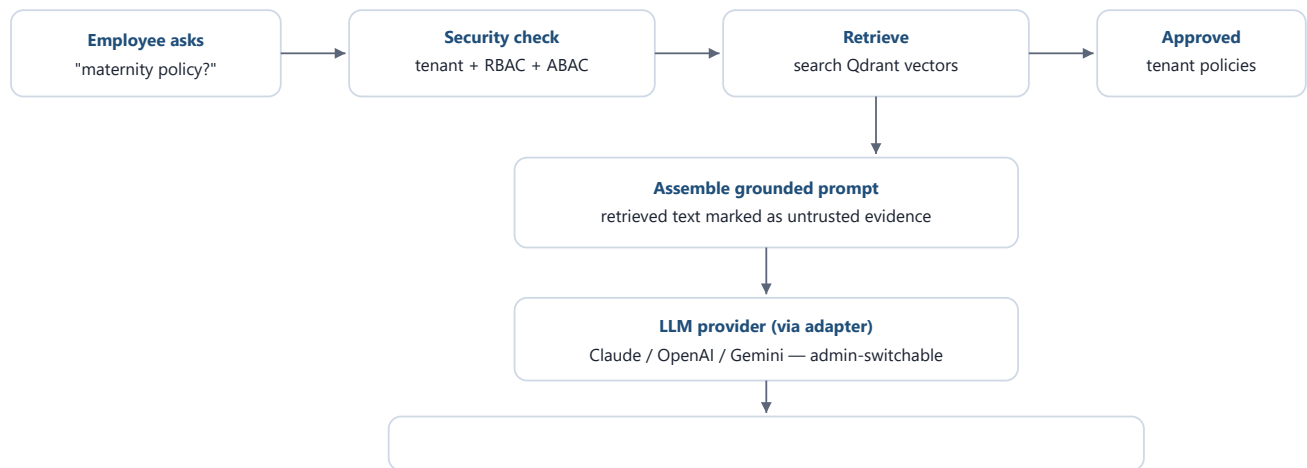


Figure 12.1 — RAG flow. LLM = Large Language Model. The AI reads only what the user is allowed to read, and cites its sources.

The non-negotiable AI principles

- **Grounded answers only** — cite approved sources; if evidence is weak or conflicting, *refuse or escalate* rather than guess.
- **Tenant isolation** — every retrieval, memory, cache and log is tenant-scoped; cross-tenant access is impossible and continuously tested.
- **Permission-aware** — the AI sees only what the *current user* is allowed to see, re-checked on every request.
- **No direct AI mutations** — the AI can *propose* an action but can never change data itself; a human takes the action through the normal secured path.
- **Provider independence** — models sit behind interfaces; the admin can switch provider (Claude/OpenAI/Gemini) without code changes.
- **Privacy** — your data is used to answer *your* questions only, and is never used to train shared models.

The supporting decisions (ADR-030 to 035)

Concern	How it is handled
Where vectors live	Self-hosted Qdrant first (no paid subscription); Azure AI Search optional later — switchable through an interface.
Watching the AI	OpenTelemetry-based observability with privacy-safe metrics (ADR-031).
Chat memory	Session-only by default; long-term memory is opt-in and encrypted (ADR-032).

Cost control	Per-tenant budgets, quotas and hard limits so spend can never run away (ADR-033).
Quality & fairness	A formal evaluation framework — including bias checks for employment use — gates every release (ADR-034).
Speed & cost cache	Self-hosted Redis semantic cache, deny-by-default for personal HR/payroll answers, re-authorised on every hit (ADR-035).

DELIBERATE GUARDRAIL

The AI is intentionally **read-only** and cannot make final employment, payroll, promotion or termination decisions. For any high-impact use, a human must meaningfully review — never rubber-stamp. This reflects emerging regulation (e.g. the EU AI Act treats worker-management AI as high-risk).

13. Glossary of Terms

Every short form used in this handbook and the source documents, expanded.

Term	Full form / meaning
ABAC	Attribute-Based Access Control — permissions based on attributes (department, location) as well as role.
ADR	Architecture Decision Record — a short record of one design decision and its rationale.
AI	Artificial Intelligence.
API	Application Programming Interface — the contract by which software talks to other software.
ATS	Applicant Tracking System — recruitment software.
BI	Business Intelligence — analytics and reporting.
CTC	Cost To Company — an employee's total salary package.
DDD	Domain-Driven Design — modelling software around the business domain.
DPDP	Digital Personal Data Protection (Act) — India's data-protection law.
DR / RPO / RTO	Disaster Recovery / Recovery Point Objective / Recovery Time Objective.
EF	Entity Framework — .NET's data-access library.
ESI	Employees' State Insurance — Indian statutory health insurance contribution.
ESS	Employee Self-Service — employees managing their own profile, leave, payslips.
FBP	Flexible Benefits Plan — employee-chosen tax-efficient salary components.
FNF / FFS	Full & Final Settlement — final dues calculation when an employee exits.
FR	Functional Requirement.
HCM	Human Capital Management — a broader term for enterprise HR suites.
HRMS	Human Resource Management System.
JWT	JSON Web Token — a signed token proving a user's identity.
LMS	Learning Management System — training/courses.
LLM	Large Language Model — the AI model behind the assistant.
LWF	Labour Welfare Fund — an Indian statutory contribution.
MFA	Multi-Factor Authentication.
MoSCoW	Must / Should / Could / Won't — a prioritisation method.
OKR	Objectives & Key Results — a goal-setting framework.
OpenAPI	The industry-standard format for describing a REST API.

OWASP	Open Worldwide Application Security Project — publishes the "Top 10" web risks.
PEPM	Per Employee Per Month — common SaaS pricing unit.
PF	Provident Fund — Indian statutory retirement contribution.
PII	Personally Identifiable Information.
PMS	Performance Management System.
PRD	Product Requirements Document.
PT	Professional Tax — an Indian state-level tax.
RAG	Retrieval-Augmented Generation — AI that answers from retrieved approved documents, with citations.
RBAC	Role-Based Access Control — permissions based on a user's role.
RLS	Row-Level Security — the database itself blocks rows from other tenants.
SaaS	Software as a Service — software delivered over the internet by subscription.
SLA	Service Level Agreement — a committed response/resolution time.
SMB	Small & Medium Business.
SOLID / DRY / KISS / YAGNI	Software-design principles for clean, maintainable code.
SSO / SCIM / OIDC	Single Sign-On / user-provisioning standard / OpenID Connect — enterprise login standards.
STRIDE	A threat-modelling method: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege.
TDS	Tax Deducted at Source — Indian income-tax withholding.
WAF	Web Application Firewall.
WCAG	Web Content Accessibility Guidelines.

14. Index of All 41 Approved Documents

The authoritative source documents behind this handbook. All carry Approved status.

Constitution & strategy

Document	Area
HRMS Master Plan	Vision
Architecture Principles	Architecture
Competitor Analysis (India, v1.3)	Market
Competitor Gap Analysis	Market
Product Owner Positioning Hypothesis	Strategy
PRD — Platform Phased Delivery (v1.1)	Product
Roadmap — Phased Delivery	Roadmap

Architecture, database & security

Document	Area
Architecture & Product Review — Platform	Architecture
Provider-Agnostic Architecture Review	Architecture
Phase 3 Final Review — Architecture Gate	Architecture
Database Design — Platform Foundations (v1.2)	Database
Database Design — Tenant Catalog + RLS	Database
Database Design — Identity & Access	Database
Security Design & Threat Model	Security

UI/UX & API

Document	Area
UI Design System (DESIGN-SYSTEM-001)	UI/UX
UI Screen Specs (SCREENS-001)	UI/UX
UI Design Spec v2.1 + prototype (DESIGN-SPEC-002)	UI/UX
Phase 4 UI Architecture Alignment Checkpoint	UI/UX
API Specification — Foundational Endpoints (v1.2)	API
OpenAPI YAML — Foundational v1 (1.2.0)	API

Phase 5 Review — Foundational OpenAPI Package	API
---	-----

AI strategy & decisions (ADRs)

Document	Area
AI Strategy — Enterprise RAG & Prompts (v2.1)	AI
ADR-001 React/Next.js · ADR-002 .NET · ADR-003 SQL Server · ADR-004 Modular Monolith	Decisions
ADR-005 Multi-tenancy · ADR-006 Tenant context · ADR-007 Effective-dating	Decisions
ADR-008 Identity · ADR-009 Event backbone · ADR-010 Workflow · ADR-011 Rule engine	Decisions
ADR-019 AI/RAG architecture · ADR-027 Provider abstraction	Decisions
ADR-030 Vector store · ADR-031 AI observability · ADR-032 Conversation memory	Decisions
ADR-033 AI cost governance · ADR-034 RAG evaluation · ADR-035 Semantic cache	Decisions

NOTE ON SCOPE

Several documents are still in *Draft* and were deliberately excluded from this handbook because they are not yet approved — for example the AI Operations Handbook, the AI Security Extension, the Phase-1 review refresh, and the per-feature specification sets (Tenant, Identity, Leave). They will be folded into a future revision once approved.